

05 장 학습 과제

과제 1. 동적 배열의 깊은 복사

정수 배열을 동적으로 관리하는 **IntArray** 클래스를 작성한다.

필수 데이터 멤버

```
std::size_t size;
int* data;
```

필수 기능

- 인수로 전달한 크기만큼 동적 배열 생성
- 모든 원소를 0으로 초기화
- 소멸자에서 배열 해제
- **Set()**과 **Get()**으로 원소 변경 및 조회
- **GetSize()**로 배열 크기 반환
- 복사 생성자에서 깊은 복사
- 복사 대입 연산자에서 깊은 복사
- 자기 대입 안전성 보장
- 서로 다른 크기의 배열 대입 지원

```
// 확인 코드
IntArray first(3);
first.Set(0, 10);
first.Set(1, 20);
first.Set(2, 30);

IntArray second = first;
second.Set(0, 100);

std::cout << first.Get(0) << '\n';
std::cout << second.Get(0) << '\n';

IntArray third(10);
third = first;

std::cout << third.GetSize() << '\n';

third = third;
std::cout << third.Get(1) << '\n';
```

원본과 복사본은 서로 다른 배열을 소유해야 한다. 자기 대입 후에도 데이터가 유지되어야 하며 프로그램 종료 시 이중 해제가 발생하지 않아야 한다.

과제 2. 이동을 지원하는 **ImageBuffer**

픽셀 데이터를 동적으로 저장하는 **ImageBuffer** 클래스를 작성한다.

상태

- 이미지 너비
- 이미지 높이
- 픽셀 수만큼 동적 할당한 `unsigned int` 배열

필수 생성자

```
ImageBuffer(int width, int height);
```

너비와 높이가 0 이하이면 빈 이미지로 생성한다.

필수 특별 멤버 함수

- 소멸자
- 복사 생성자
- 복사 대입 연산자
- 이동 생성자
- 이동 대입 연산자

이동 생성자와 이동 대입 연산자에는 `noexcept`를 지정한다. 이동 후 원본 객체는 너비와 높이가 0이고 픽셀 포인터가 `nullptr`인 상태로 만든다.

```
// 필수 멤버 함수
int GetWidth() const;
int GetHeight() const;
bool IsEmpty() const;
void SetPixel(int x, int y, unsigned int color);
unsigned int GetPixel(int x, int y) const;
```

좌표가 범위를 벗어나면 `SetPixel()`은 아무 작업도 하지 않고 `GetPixel()`은 0을 반환한다.
확인 항목

- 복사본의 픽셀을 변경해도 원본이 유지되는가
- 이동 후 대상 객체가 원래 픽셀을 소유하는가
- 이동 후 원본에 새로운 이미지를 대입할 수 있는가
- 자기 복사 대입과 자기 이동 대입에서 오류가 발생하지 않는가
- 프로그램 종료 시 이중 해제와 메모리 누수가 없는가

과제 3. 복사와 이동 호출 추적

특별 멤버 함수가 호출될 때 함수 이름과 객체 번호를 출력하는 `TraceObject` 클래스를 작성한다.

출력할 함수

- 기본 생성자
- 매개변수가 있는 생성자
- 복사 생성자
- 복사 대입 연산자
- 이동 생성자
- 이동 대입 연산자
- 소멸자

```
class TraceObject
{
private:
    int id;
};
```

다음 코드를 실행하고 호출 순서를 기록한다.

```
TraceObject first(1);
TraceObject second = first;
TraceObject third = std::move(first);

second = third;
third = TraceObject(4);
```

값을 반환하는 함수도 작성한다.

```
TraceObject CreateObject(int id);
```

Debug와 Release 구성에서 실행 결과를 비교한다. 복사 생략은 C++ 언어 규칙과 컴파일러의 선택에 영향을 받으므로, 출력 횟수가 예상과 다를 수 있는 이유를 설명한다.

과제 4. 복사 정책 분석

각 클래스에 적합한 복사 정책을 선택하고 이유를 작성한다.

1. 게임 좌표를 저장하는 **Position**
2. 하나의 파일 핸들을 소유하는 **FileHandle**
3. 플레이어를 가리키는 화면 표시용 **PlayerView**
4. 여러 객체가 공동으로 사용하는 텍스처 리소스
5. 독립된 픽셀 데이터를 가져야 하는 **Image**
6. 동시에 하나의 스레드만 소유해야 하는 잠금 객체

선택할 수 있는 정책은 다음과 같다.

- 깊은 복사
- 공유 소유
- 비소유 참조
- 복사 금지
- 이동 전용

각 타입의 복사 생성자와 복사 대입 연산자를 = default, = delete 또는 사용자 정의 구현 중 어떤 방식으로 구성할지도 함께 작성한다.

과제 5. Rule of Zero를 적용한 캐릭터 인벤토리

직접 동적 메모리를 사용하지 않고 std::string과 std::vector를 사용하여 CharacterInventory 클래스를 작성한다.

상태

- 캐릭터 이름
- 골드

- 아이템 이름 목록

```
//필수 기능
void AddItem(const std::string& item);
bool RemoveItem(const std::string& item);
bool Contains(const std::string& item) const;
void AddGold(int amount);
void Print() const;
```

소멸자, 복사 생성자, 복사 대입 연산자, 이동 생성자와 이동 대입 연산자를 직접 작성하지 않는다.

원본 인벤토리를 복사한 뒤 복사본에서 아이템을 추가하거나 제거해도 원본 목록은 유지되어야 한다. 시작 아이템을 가진 인벤토리를 값으로 반환하는 함수도 작성한다.

```
CharacterInventory CreateStarterInventory(const std::string&
characterName);
```

함수에서 지역 객체를 만들고 값으로 반환한다. 반환문에는 `std::move()`를 사용하지 않는다.

과제 6. 이동 전용 로그 세션

하나의 로그 파일 연결을 소유한다고 가정한 `LogSession` 클래스를 설계한다. 실제 파일 입출력 대신 세션 번호와 열림 상태만 관리해도 된다.

설계 조건

- 생성자에서 세션을 열린 상태로 만듦
- 소멸자에서 세션을 닫음
- 복사 생성자 삭제
- 복사 대입 연산자 삭제
- 이동 생성자 허용
- 이동 대입 연산자 허용
- 이동 후 원본은 닫힌 상태로 변경
- `IsOpen()`으로 상태 확인

```
LogSession first(1);
// LogSession second = first; // 컴파일 오류
LogSession second = std::move(first);
```

복사가 금지되는 이유와 이동은 허용할 수 있는 이유를 코드 주석으로 설명한다. 함수에서 `LogSession`을 값으로 반환하고 복사 생략 또는 이동을 통해 호출자에게 전달되는 과정도 확인한다.